



THE UNIVERSITY OF
SYDNEY

THE UNIVERSITY OF SYDNEY

AERO4260 AERODYNAMICS 2

SEMESTER 1 2024

MAJOR ASSIGNMENT

Scramjet Engine Design and Optimisation

Author: Thomas BULLOCK

SID: 530754154

simple extension granted

May 31, 2024

Contents

Nomenclature	iii
1 Introduction	1
2 Literature Review	1
3 Methodology	2
3.1 Freestream Conditions	2
3.2 Stagnation Conditions	2
3.3 Inlet	2
3.4 Isolator	4
3.5 Combustor	5
3.6 Nozzle	7
4 Benchmark Analysis	8
5 Optimisation	9
5.1 Inlet	9
5.1.1 Deflection set 2	9
5.1.2 Deflection set 3	9
5.2 Isolator	10
5.3 Combustor	10
5.4 Nozzle	11
5.5 Optimised Scramjet	12
5.5.1 Deflection set 2	12
5.5.2 Deflection set 3	13
6 Discussion	14
7 Conclusion	14
References	xv
A Code	xv
A.1 Main	xv
A.2 Freestream Conditions	xvii

A.3 Stagnation Conditions	xviii
A.4 Inlet	xix
A.5 Isolator	xx
A.6 Combustor	xxi
A.7 Nozzle	xxii

Nomenclature

Symbols			
β	wave angle [°]	T_∞	freestream static temperature [K]
γ_∞	freestream air heat capacity ratio [-]	$T_{t,\infty}$	freestream stagnation temperature [K]
γ_{com}	combustor heat capacity ratio [-]	Subscripts	
γ_{iso}	isolator heat capacity ratio [-]	0	freestream or inlet entrance
γ_{noz}	nozzle heat capacity ratio [-]	1	inlet exit or isolator entrance
Θ	nozzle expansion angle [°]	2	isolator exit
θ	inlet deflection angle [°]	3	combustor inlet
F	Thrust [N]	4	combustor exit
h	flight altitude [m]	5	nozzle inlet
I_{spW}	Weight specific impulse [s]	9	nozzle exit
I_{sp}	Specific impulse [kN kg ⁻¹ s ⁻¹]	i+1	upstream of shock
M_∞	flight Mach number [-]	i	downstream of shock
p_∞	freestream static pressure [Pa]	Superscript	
p_a	sea level static pressure [Pa]	-	Flow property within an element
$p_{t,\infty}$	freestream stagnation pressure [Pa]		

1 Introduction

A supersonic combustion ramjet (scramjet) is a type of air breathing engine designed for hypersonic flight, typically speeds greater than Mach 5. A scramjet engine includes an inlet, isolator, combustor and a diverging nozzle. They work by compressing the air through the intake, which is possible due to the supersonic speed in which they travel. For this reason, scramjets do not have compressor or turbine or any moving parts at all. The flow through a scramjet remains supersonic at all times, which is what differentiates it from a ramjet. Scramjets serve a unique purpose in that they are designed to fill the gap between traditional turbomachinery based engines and rockets. Scramjets can operate at hypersonic speeds while not having to carry oxidiser. This makes them an obvious choice for hypersonic flight within the atmosphere.

This report sets out a one dimensional analysis on the performance of a scramjet, aiming to optimise specific impulse through various methods. Simplifications of various components have been made to aid in the analysis without compromising the accuracy too much.

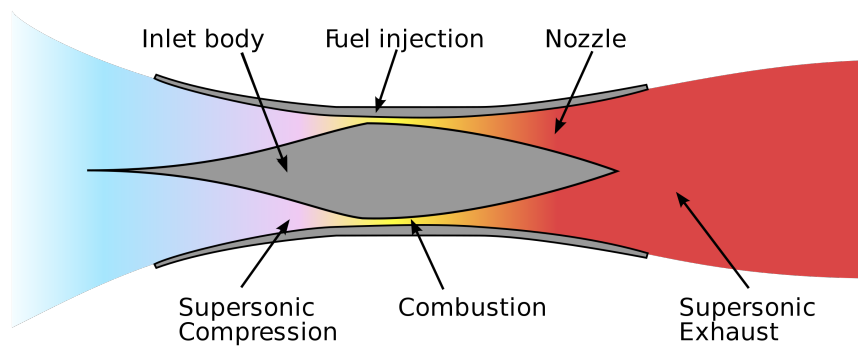


Figure 1: Scramjet geometry

2 Literature Review

Typical operating conditions were explored in the literature to give the analysis a sense of direction. Scramjet engines typically have a specific impulse, I_{spW} , in the range of 1000s to 4000s [1]. Operating temperatures within a scramjet are between 800 K - 3000 K [2].

Scramjets have a relatively short history, the first successful flight test only happened in 1991. It wasn't until the 21st century that significant developments in hypersonic technology were made. Scramjets have been used on experimental aircraft such as NASA X-43, which set the record for the fastest jet powered aircraft at Mach 9.3. There isn't much funding for these types of projects anymore due to the practicality of flying at hypersonic speeds. Since, the use of scramjets have been on weapons systems such as missiles [3].

3 Methodology

3.1 Freestream Conditions

The freestream conditions were given as follows:

$$T_{\infty} = \begin{cases} 288.19 - 0.00649 \times h & h < 11000 \\ 216.69 & 11000 \leq h \leq 25000 \\ 141.94 + 0.00299 \times h & h \geq 25000 \end{cases}$$

$$p_{\infty} = \begin{cases} 1.01 \times 10^5 \times \left(\frac{T_{\infty}}{288.19} \right)^{5.256} & h < 11000 \\ 2.27 \times 10^4 \times e^{(1.73 - 0.000157h)} & 11000 \leq h \leq 25000 \\ 2.49 \times 10^3 \times \left(\frac{T_{\infty}}{216.6} \right)^{-11.388} & h \geq 25000 \end{cases}$$

The hypersonic vehicle that the scramjet powers cruises at an altitude, h , of between 25 km and 30 km. For the initial analysis, it was assumed the hypersonic vehicle was cruising at 27.5 km.

The code used for the freestream conditions can be seen in appendix A.2.

3.2 Stagnation Conditions

The stagnation conditions were calculated using Equation 1 and 2.

$$T_{t,\infty} = \left(1 + \frac{\gamma_{\infty} - 1}{2} M_{\infty}^2 \right) T_{\infty} \quad (1)$$

$$p_{t,\infty} = \left(1 + \frac{\gamma_{\infty} - 1}{2} M_{\infty}^2 \right)^{\frac{\gamma_{\infty}}{\gamma_{\infty} - 1}} p_{\infty} \quad (2)$$

The code takes the freestream conditions calculated in section 3.1 as an input and outputs the resulting stagnation pressure and temperature. The flight Mach number, $M_{\infty} = 8.4$, was given as per my SID number. The stagnation condition code can be seen in section A.3.

3.3 Inlet

The inlet of the scramjet was simplified to a four oblique shock model. The inlet was required to reduce the Mach number of the incoming flow from $M_{\infty} = 8.4$ to $M_1 = 3.5 - 4$. The method implemented was an oblique shock calculation using the analytical solution,

completed four times for each deflection angle. The analytical calculation for the first deflection angle will be explained for clarity.

The $\theta - \beta - M$ relation, shown in Equation 3, was used over a wide range of wave angles (β) to output a range of corresponding deflection angles (θ), for a given Mach number.

$$\tan \theta = 2 \cot \beta \left[\frac{M_i^2 \sin^2 \beta - 1}{M_i^2 (\gamma_\infty + \cos 2\beta) + 2} \right] \quad (3)$$

As only weak shock solutions are considered, if the deflection angle in the inlet was larger than the maximum deflection angle obtained from Equation 3, the result was deemed not possible. The code dealt with this by throwing an error message and the flow properties were set NaN.

Provided the deflection angle in the inlet was possible, the actual wave angle (β) was determined using the $\beta - \theta - M$ relation [4]:

$$\tan \beta = \frac{M_i^2 - 1 + 2\lambda \cos \left[\frac{4\pi\delta + \cos^{-1}\chi}{3} \right]}{3 \left(1 + \frac{\gamma_\infty - 1}{2} M_i^2 \right) \tan \theta} \quad (4)$$

where $\delta = 0$ yields the strong shock solution and $\delta = 1$ yields the weak shock solution. As strong shock solutions aren't considered in the inlet, $\delta = 1$.

Where λ and χ were determined from the following relations:

$$\lambda = \left[(M_i^2 - 1)^2 - 3 \left(1 + \frac{\gamma_\infty - 1}{2} M_i^2 \right) \left(1 + \frac{\gamma_\infty + 1}{2} M_i^2 \right) \tan^2 \theta \right]^{1/2} \quad (5)$$

$$\chi = \frac{(M_i^2 - 1)^3 - 9 \left(1 + \frac{\gamma_\infty - 1}{2} M_i^2 \right) \left(1 + \frac{\gamma_\infty + 1}{2} M_i^2 + \frac{\gamma_\infty + 1}{4} M_i^4 \right) \tan^2 \theta}{\lambda^3} \quad (6)$$

Once the wave deflection angle was calculated, the downstream conditions of the oblique shock were then be calculated.

The Mach number normal to the oblique shock was given by the relation:

$$M_{iN} = M_i \sin(\beta) \quad (7)$$

This result was used in combination with the normal shock relations to determine the pressure and temperature downstream of the oblique shock.

The downstream normal Mach number was then calculated using the relation:

$$M_{i+1N}^2 = M_{iN}^2 \left(\frac{(\gamma_\infty - 1)M_{iN} + 2}{2\gamma_\infty M_{iN} - (\gamma_\infty - 1)} \right) \quad (8)$$

$$M_{i+1} = \frac{M_{i+1N}}{\sin(\beta - \theta)} \quad (9)$$

The downstream temperature was then calculated using the relation:

$$\frac{T_{i+1}}{T_{t,i}} = \left[1 + \frac{2\gamma_\infty}{\gamma_\infty + 1} (M_{iN}^2 - 1) \right] \left[\frac{2 + (\gamma_\infty - 1)M_{iN}^2}{(\gamma_\infty + 1)M_{iN}^2} \right] \quad (10)$$

The downstream pressure was then calculated using the relation:

$$\frac{p_{i+1}}{p_{t,i}} = 1 + \frac{2\gamma_\infty}{\gamma_\infty + 1} (M_{iN}^2 - 1) \quad (11)$$

This process was put into a FOR loop within MATLAB to repeat for each ramp angle. The final output was T_1 , p_1 and M_1 which were the flow conditions at the inlet exit or isolator entrance. The code can be seen in appendix A.4.

3.4 Isolator

The isolator in a scramjet plays a critical role in increasing the static pressure of the flow, as well as ensuring the combustion process doesn't interfere with the inlet. The flow is complex to analyse and so simplifications were made which bypass the need to analyse such complex flows without losing too much accuracy. The downstream properties of the final oblique shock were treated as the flow properties of the isolator entrance.

The process started by calculating the total pressure and the isolator entrance, given by the following formula:

$$p_{t,1} = p_1 \left(1 + \frac{\gamma_{iso} - 1}{2} M_1^2 \right)^{\frac{\gamma_{iso}}{\gamma_{iso} - 1}} \quad (12)$$

The total pressure recovery was used in combination with the result from Equation 12 to obtain the total pressure at station 2.

$$p_{t,2} = p_{t,1} \times \frac{p_{t,2}}{p_{t,1}} \quad (13)$$

The static pressure at station 2 was then calculated using the pressure ratio between stations 1 and 2:

$$p_2 = p_1 \times \frac{p_2}{p_1} \quad (14)$$

With the results from Equations 13 and 14, the Mach number at the isolator exit was then calculated using the following formula:

$$M_2 = \sqrt{\frac{2}{\gamma_{iso} - 1} \left(\left(\frac{p_{t,2}}{p_2} \right)^{\frac{\gamma_{iso} - 1}{\gamma_{iso}}} - 1 \right)} \quad (15)$$

The static temperature was calculated by using the fact that the total temperature of the flow is constant through the isolator:

$$T_{t1} = T_{t2} \quad (16)$$

$$T_{t1} = T_1 \left(1 + \frac{\gamma_{iso} - 1}{2} M_1^2 \right) \quad T_{t2} = T_2 \left(1 + \frac{\gamma_{iso} - 1}{2} M_2^2 \right) \quad (17)$$

$$T_2 = T_1 \frac{1 + \frac{\gamma_{iso} - 1}{2} M_1^2}{1 + \frac{\gamma_{iso} - 1}{2} M_2^2} \quad (18)$$

The density of the flow was calculated using the equation of state:

$$\rho_2 = \frac{p_2}{R T_2} \quad (19)$$

The velocity was then calculated using the following relation:

$$V_2 = M_2 \sqrt{\gamma_{iso} R T_2} \quad (20)$$

Finally, the mass flow rate at the isolator exit was calculated using the Equation 21:

$$\dot{m}_2 = \rho_2 A_2 V_2 \quad (21)$$

All the flow conditions at the isolator exit were evaluated, and the code can be seen in appendix A.5.

3.5 Combustor

As there is no compressor in a scramjet, all the flow properties at station 2 equal the flow properties at station 3.

The thrust output from the combustor was calculated. Mathematically, the thrust is the pressure integral along the combustor represented by Equation 22.

$$F_{com} = \left(\int_3^4 p ds \right)_x \quad (22)$$

This was implemented in MATLAB by discretising the combustor domain into N elements, one of which is seen in Figure 2.

Where θ is the combustor expansion angle given as 1° , and dx and dh are distances.

The contribution of thrust from any particular element is given by the calculation:

$$\bar{F}_{com} = (\bar{p} \cdot \tan(\theta) \cdot dx \cdot 0.07) \cdot 2 \quad (23)$$

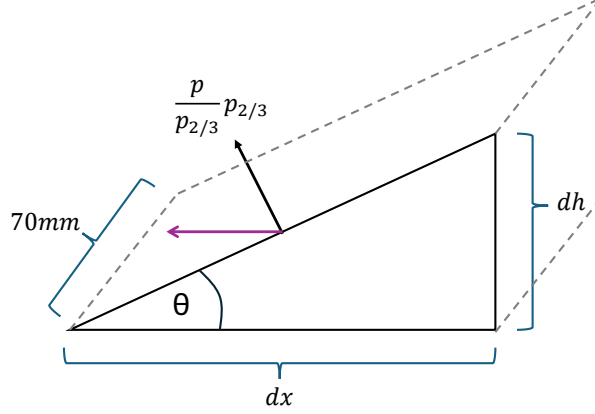


Figure 2: Element of the discretised combustor

Where $\bar{p}$ is the pressure in a particular element calculated using the pressure distribution in Figure 4 and the static pressure from Equation 14.

The total thrust contribution from the combustor was then calculated by summing the individual thrust contributions from each element. A factor of 2 was applied to account for the expansion on both sides of the combustor.

$$F_{com} = \sum_{i=1}^N \bar{F} \quad (24)$$

Where N is the number of discretised elements. The area of the combustor exit as well as the pressure was also obtained. Implementation of the method is shown in appendix A.6

The total temperature at the combustor exit was calculated by conducting an energy balance, it was assumed the combustor efficiency of $\eta_{cc} = 1$:

$$T_{t4} = \frac{C_{p,c}(T_{t3} - 288) + \eta_{cc} \cdot \text{FAR} \cdot \text{LHV}}{(1 + \text{FAR}) \cdot C_{p,h}} + 288 \quad (25)$$

The static pressure at the combustor exit was calculated using the following relation:

$$T_4 = T_{t,4} \cdot \left(1 + \frac{\gamma_{com} - 1}{2} \cdot M_4^2 \right)^{-1} \quad (26)$$

The total pressure at the combustor exit was given by the relation:

$$p_{t,4} = p_4 \left(1 + \frac{\gamma_{com} - 1}{2} M_4^2 \right)^{\frac{\gamma_{com}}{\gamma_{com} - 1}} \quad (27)$$

Where p_4 was calculated using the final pressure ratio in Figure 4.

3.6 Nozzle

The nozzle thrust was calculated using Equation 28:

$$F_{noz} = \left(\int_5^9 p \, ds \right)_x \quad (28)$$

The nozzle thrust was calculated in a similar way to the combustor, although the pressure for a given element was found through different means.

The Mach-area relationship was used, shown in Equation 29, to calculate the value of A^* . The value of A^* remains constant in the nozzle due to the isentropic condition.

$$\left(\frac{A}{A^*} \right)^2 = \frac{1}{M^2} \left[\frac{2}{\gamma + 1} \left(1 + \frac{\gamma - 1}{2} M^2 \right) \right]^{\frac{\gamma + 1}{\gamma - 1}} \quad (29)$$

As the flow is already supersonic, A^* can be thought of as an imaginary throat in the engine and was calculated using the known flow properties at the combustor exit\|nozzle inlet.

The nozzle was then discretised into N elements. Each element of the nozzle has a different value of $\frac{A}{A^*}$, due to the expansion angle. By using the SOLVE function in MATLAB on Equation 29, the Mach number for that particular element was found.

Using the isentropic relation, the pressure for a given element was then calculated:

$$\frac{\bar{p}}{p_{t,4}} = \left(1 + \frac{\gamma_{noz} - 1}{2} \bar{M}^2 \right)^{-\frac{\gamma_{noz}}{\gamma_{noz} - 1}} \quad (30)$$

Where $\bar{M}$ and $\bar{p}$ are the Mach number and pressure for a given element, respectively.

The thrust contribution from an element was then calculated in a similar fashion to the combustor, although with a different expansion angle:

$$\bar{F}_{noz} = (\bar{p} \cdot \tan(\Theta) \cdot dx \cdot 0.07) \cdot 2 \quad (31)$$

The total thrust from the nozzle was calculated by summing the thrust contribution from each element:

$$F_{noz} = \sum_{i=1}^N \bar{F}_{noz} \quad (32)$$

Where N is the number of discretised elements. Implementation of the method in MATLAB is shown in appendix A.7

The total thrust was found via the sum of the combustor thrust and nozzle thrust, shown in Equation 33

$$F_{total} = F_{com} + F_{noz} \quad (33)$$

The mass flow rate of fuel was calculated using the FAR ratio provided, as well as the mass flow rate of air, calculated in Equation 21.

$$\dot{m}_f = FAR \cdot \dot{m} \quad (34)$$

Finally, the specific impulse of the scramjet was calculated using the relation:

$$I_{sp} = \frac{F_{total}}{\dot{m}_f} \quad \text{also defined as} \quad I_{spW} = \frac{F_{total}}{\dot{m}_f \cdot g} \quad (35)$$

4 Benchmark Analysis

Prior to optimisation, an initial analysis was carried out to act as a comparison for the optimised engine.

The flight altitude was assumed constant across all tests, $h = 27.5$ km.

The initial set of deflection angles were chosen at random to be:

$$\theta = [9 \quad 8 \quad 7 \quad 6]^\circ \quad (36)$$

The total pressure recovery and pressure ratio across the isolator was selected to be:

$$\frac{p_{t,2}}{p_{t,1}} = 0.3 \quad \frac{p_2}{p_1} = 1.9 \quad (37)$$

A cavity distance of $\Delta X = 340$ mm was chosen at random.

The nozzle expansion angle was selected to be $\Theta = 2^\circ$.

Using these inputs and the methodology explained in section 3, the results were the following:

$$F_{total} = 2.84 \text{ kN} \quad (38)$$

$$I_{sp} = 12.8 \text{ kN kg}^{-1} \text{ s}^{-1} \quad (39)$$

$$I_{spW} = 1290 \text{ s} \quad (40)$$

Despite this initial result being in the lower range of expected specific impulse, discussed in section 2, it was confirmation that the methodology and its implementation were correct.

Table 1: Flow properties of the benchmark analysis

	Station Number				
	∞	1	2	4	9
Mach No. (M) [-]	8.4	3.85	2.63	1.20	1.80
Total pressure (p_t) [MPa]	22.6	13.8	4.1	2.0	2.0
Static pressure (p) [kPa]	1.68	102	194	849	358
Total temperature (T_t) [K]	3387	3387	3327	4895	4895
Static temperature (T) [K]	224	855	1390	2187	1740

5 Optimisation

Following the benchmark analysis, an optimisation study was conducted to maximise the specific impulse of the scramjet.

5.1 Inlet

Two deflection sets were tested, one of which minimised the total pressure loss at the inlet. The other maximised the total pressure loss at the inlet.

5.1.1 Deflection set 2

The four deflection angles were optimised to reduce the total pressure loss between the freestream and the inlet exit while keeping the inlet Mach number within the required range. An optimisation code was used to determine the deflection angles which resulted in the lowest total pressure loss. This resulted in the following set of deflection angles:

$$\theta = [4.43 \quad 5.97 \quad 7.53 \quad 10.56]^\circ \quad (41)$$

5.1.2 Deflection set 3

A similar procedure was undertaken for the third set of deflection angles, however they were optimised to maximise the total pressure loss. The set deflection angles which resulted in the highest total pressure loss, therefore highest static pressure gain, were calculated to be:

$$\theta = [10 \quad 9 \quad 8 \quad 7]^\circ \quad (42)$$

5.2 Isolator

The optimal isolator configuration was determined by simulating the benchmark analysis for all possible values of total pressure recovery and the pressure ratio. The specific impulse, I_{spW} , was then contoured to produce Figure 3

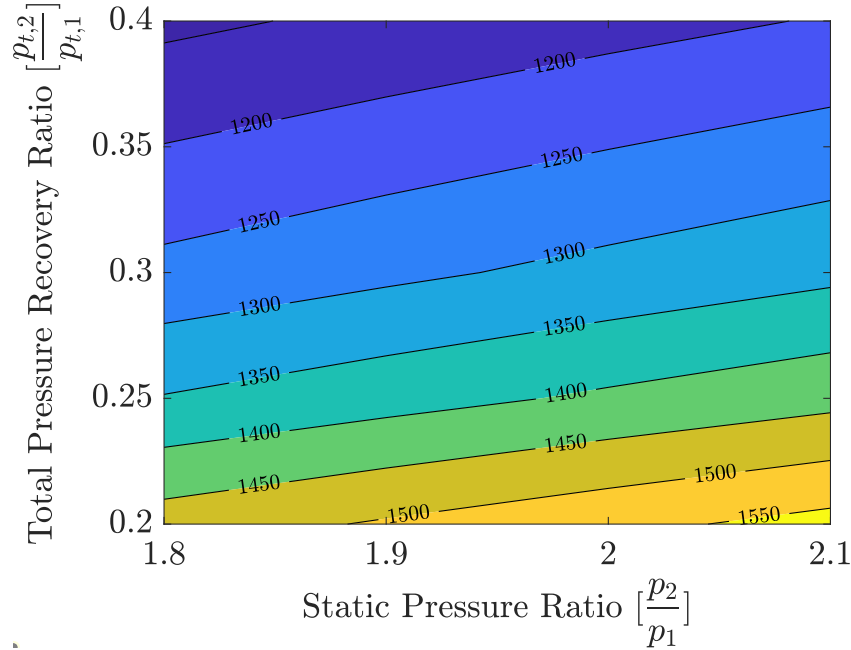


Figure 3: Static pressure ratio and total pressure recovery contoured by specific impulse I_{spW} for the benchmark test

From this optimisation study, it was concluded that the optimal setting for static pressure ratio and total pressure recovery to achieve the highest specific impulse was 2.1 and 0.2, respectively.

5.3 Combustor

Figure 4 shows the relationship between the pressure ratio along the combustor for various different cavity distances. It was evident from Figure 4, that a cavity distance of $\Delta X = 40$ mm would result in an inlet unstart as the pressure ratio at the start of the combustor is much higher than the isolator exit. Essentially, this means that $p_2 \neq p_3$, the pressure waves would propagate upstream, resulting in the engine losing thrust.

Another conclusion that was made using Figure 4 was the cavity position that would result in the most thrust. The thrust from the combustor is proportional to the integral of the data in Figure 4. The TRAPZ function was used in MATLAB to calculate the integral of the pressure ratio through the combustion chamber for each of the cavity distances. The results of which are shown in Table 2.

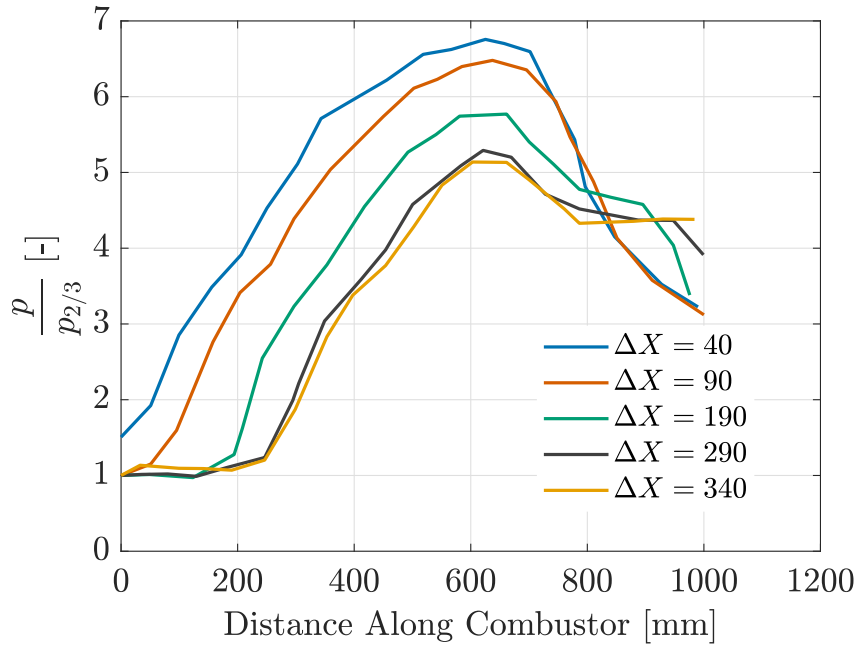


Figure 4: Pressure distributions within the combustor at various cavity distances

Table 2: Integral of the pressure ratios for various cavity distances

Cavity Distance (ΔX) [mm]	Evaluated Integral [$1 \times 10^3 \text{ mm}^2$]
40	4.81
90	4.43
190	3.73
290	3.38
340	3.25

The unit of the integral evaluated in Table 2 doesn't have any physical meaning, but because it's proportional to the thrust produced, it was used to evaluate the optimal cavity distance. Table 2 shows that a cavity distance of $\Delta X = 40$ mm would result in the largest thrust output. The conclusions made previously, however, mean that this cavity distance isn't possible due to inlet unstart.

The optimal cavity distance for thrust is therefore $\Delta X = 90$ mm.

5.4 Nozzle

The optimal nozzle expansion angle was determined by simulating the benchmark scramjet, seen in section 4, for all possible expansion angle values. The relationship between expansion angle and weight specific impulse is shown in Figure 5.

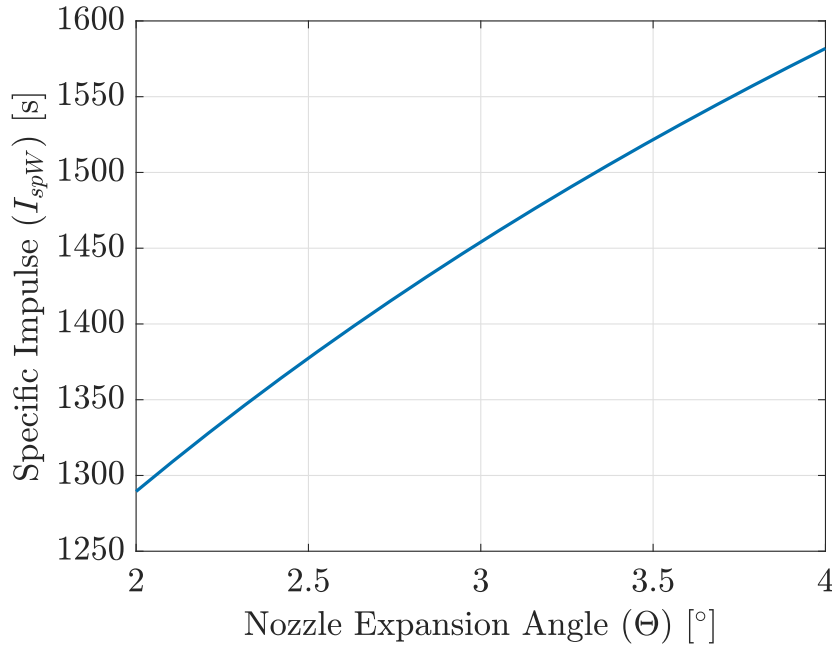


Figure 5: The relationship between nozzle expansion angle and specific impulse for the benchmark study

It is clear from Figure 5 that a larger nozzle expansion angle results in a larger specific impulse. From this optimisation study, it was concluded that the optimal nozzle expansion angle is ($\Theta = 4^{\circ}$)

5.5 Optimised Scramjet

The optimised scramjet was then analysed using the two sets of deflection angles presented in section 5.1, in combination with the conclusions presented for the other sections of the engine. These were:

- $\frac{p_{t,2}}{p_{t,1}} = 0.2$
- $\frac{p_2}{p_1} = 2.1$
- $\Delta X = 90 \text{ mm}$
- $\Theta = 4^{\circ}$

5.5.1 Deflection set 2

Using the inputs outlined and the methodology explained in section 3, the results for deflection set 2 from section 5.1.1 were the following:

$$F_{total} = 3.66 \text{ kN} \quad (43)$$

$$I_{sp} = 17.3 \text{ kN kg}^{-1} \text{ s}^{-1} \quad (44)$$

$$I_{spW} = 1770 \text{ s} \quad (45)$$

Table 3: Flow properties for deflection angle set 2

	Station Number				
	∞	1	2	4	9
Mach No. (M) [-]	8.4	4.00	2.45	1.20	2.1
Total pressure (p_t) [MPa]	22.6	16.2	3.25	1.51	1.51
Static pressure (p) [kPa]	1.68	97	203	634	166
Total temperature (T_t) [K]	3387	3387	3310	4881	4881
Static temperature (T) [K]	224	806	1503	2180	1530

5.5.2 Deflection set 3

Using the inputs outlined and the methodology explained in section 3, the results for deflection set 3 from section 5.1.2 were the following:

$$F_{total} = 5.34 \text{ kN} \quad (46)$$

$$I_{sp} = 23.3 \text{ kN kg}^{-1} \text{ s}^{-1} \quad (47)$$

$$I_{spW} = 2380 \text{ s} \quad (48)$$

Table 4: Flow properties for deflection angle set 3

	Station Number				
	∞	1	2	4	9
Mach No. (M) [-]	8.4	3.51	2.00	1.20	2.1
Total pressure (p_t) [MPa]	22.6	11.6	2.32	2.21	2.21
Static pressure (p) [kPa]	1.68	141	296	924	242
Total temperature (T_t) [K]	3387	3387	3293	4868	4868
Static temperature (T) [K]	224	978	1824	2175	1526

Table 5: Thrust and specific impulse of the three deflection sets analysed

	Deflection set		
	Benchmark	2	3
Thrust (F) [kN]	2.84	3.66	5.34
Specific Impulse (I_{sp}) [$\text{kN kg}^{-1} \text{s}^{-1}$]	12.8	17.3	23.3
Weight Specific Impulse (I_{spW}) [s]	1290	1770	2380

6 Discussion

There are some consistent patterns that emerge from Tables 1, 3 and 4.

The total temperature, T_t changes across the isolator a small amount, simply due to the change in heat capacity ratio from $\gamma_\infty = 1.4$ to $\gamma_{iso} = 1.36$. There is a large increase across the combustor due to the energy release from the fuel. It then stays constant in the nozzle due to isentropic condition. The static temperatures are consistent with the values presented in section 2, it rises through the engine before decreasing through the nozzle. The total pressure decreases in through the engine due to shock waves and a small pressure loss in the combustor. The total pressure remains constant in the nozzle, due to the isentropic condition. The static pressure rises in the engine until the combustor exit, it then decreases through the nozzle to create thrust. The decrease in static values through the nozzle occur due to the flow being accelerated.

Regarding the optimisation, the results of which are shown in Table 5, shows that deflection set 2 increased the specific impulse by 37%. Deflection set 3 increased the specific impulse by 84%. This was due to flow being slowed much more in deflection set 3, hence raising the static pressure more, producing more thrust through the engine.

7 Conclusion

A one dimensional performance analysis of a scramjet was completed, along with an optimisation study. Flow properties were calculated and tabulated at all points in the engine. The benchmark analysis confirmed that the method used and its implantation, were of sound nature, as the values obtained aligned with the literature in section 2. Following this, optimisation of the scramjet was completed and tested on two sets of deflection angles, which resulted in large increases for the specific impulse. The most optimised result was obtained by using deflection set 3, which resulted in a specific impulse of $I_{spW} = 2380 \text{ s}$ which was an 84% increase from the benchmark study.

References

- [1] D. KORS, “Design considerations for combined air breathing-rocket propulsion systems,” in *2nd International Aerospace Planes Conference*, 1990, p. 5216.
- [2] D. Andreadis, “Scramjet engines enabling the seamless integration of air & space operations,” *Pratt & Whitney Space Propulsion, Hypersonics, West Palm Beach, FL*, pp. 33 410–9600, 2004.
- [3] C. Staff. “X-51a races to hypersonic record.” (), [Online]. Available: <https://www.cnet.com/science/x-51a-races-to-hypersonic-record/>.
- [4] J. D. Anderson, *Modern Compressible Flow with Historical Perspective*, 4th. McGraw Hill Education, 2021, Published: February 11, 2020.

A Code

A.1 Main

```

1 clear;
2 clc;
3 close all;
4
5 StartupPlotting();
6
7 % Constants
8 M_infty = 8.4; % Freestream Mach number
9 gamma_infty = 1.4; % Freestream gamma
10 R = 287; % Gas constant
11
12 %% FREESTREAM CONDITIONS
13 % Calculate freestream conditions at an altitude of 27,500 meters
14 [T_infty, P_infty] = FreestreamConditions1(27500);
15
16 %% STAGNATION CONDITIONS
17 % Calculate stagnation conditions based on freestream conditions
18 [Tt_infty, Pt_infty] = StagnationConditions1(T_infty, P_infty, M_infty,
19     gamma_infty);
20
21 %% INLET
22 % Define shock angle range and deflection angles for the inlet
23 beta_array = 0.01:0.1:90;
24 theta_array = [10, 8, 8, 8]; % Deflection angles
25
26 % Calculate inlet flow properties
27 [inlet_Mach_array, inlet_temps, inlet_pressures, temp_ratio,
28     pressure_ratio, beta] = inlet1(theta_array, M_infty, gamma_infty,
29     beta_array, T_infty, P_infty);
30
31 % Display the Mach number at the inlet exit
32 fprintf('Mach Number at inlet exit = %.3f\n', inlet_Mach_array(end));
33
34 %% ISOLATOR - STATION 1 TO 2
35 % Values from inlet exit

```

```

33 M1 = inlet_Mach_array(end);
34 T1 = inlet_temps(end);
35 P1 = inlet_pressures(end);
36
37 gamma_isolator = 1.36; % Heat capacity ratio in the isolator
38
39 % Define pressure recovery and static pressure ratios for the isolator
40 iso_pt_ratio = 0.3; % Total pressure recovery ratio (pt2/pt1)
41 iso_p_ratio = 1.9; % Static pressure ratio (p2/p1)
42 iso_area = 50e-3 * 70e-3; % Cross-sectional area (m^2)
43 iso_length = 350e-3; % Length of the isolator (m)
44
45 % Calculate isolator flow properties
46 [M2, T2, P2, m2_dot] = isolator1(iso_pt_ratio, iso_p_ratio, M1, T1, P1,
    gamma_isolator, iso_area, R);
47
48 % Display the results from the isolator
49 fprintf('Mach number at isolator exit: %.2f\n', M2);
50 fprintf('Static temperature at isolator exit: %.2f K\n', T2);
51 fprintf('Static pressure at isolator exit: %.2f Pa\n', P2);
52 fprintf('Mass flow rate at isolator exit: %.2f kg/s\n', m2_dot);
53
54 %% COMBUSTOR
55 % Define combustor parameters
56 M4_array = [1.1, 1.2, 1.1, 1.05, 1.02]; % Array of Mach numbers at the
    combustor exit
57 gamma_combustor = gamma_isolator; % Heat capacity ratio in the
    combustor
58 combustor_expansion_angle = 1; % Expansion angle of the combustor (
    degrees)
59 initial_combustor_height = 50e-3; % Initial height of the combustor (m)
60 initial_combustor_width = 70e-3; % Initial width of the combustor (m)
61
62 % Load pressure distribution data for the combustor
63 CombustorPressureDistribution = load('CombustorPressureDistribution.mat
    ');
64 pressure_distributions = {'X90'}; % Specify which pressure
    distributions to use
65
66 for z = 1:length(pressure_distributions)
67     % Extract the current distribution name
68     dist_name = pressure_distributions{z};
69
70     % Extract the pressure ratios and lengths for the current
        distribution
71     com_p_ratios = CombustorPressureDistribution.(dist_name)(:, 2);
72     com_p_ratio_lengths = CombustorPressureDistribution.(dist_name)(:,
        1) * 1e-3; % Convert to meters
73
74     % Calculate the thrust generated by the combustor
75     [combustor_thrust, P4, A4] = combustor1(P2, com_p_ratios,
        com_p_ratio_lengths, initial_combustor_height,
        initial_combustor_width, combustor_expansion_angle);
76
77     % Display the total combustor thrust
78     fprintf('Total Combustor thrust for %s: %.2f N\n', dist_name,
        combustor_thrust);
79 end

```

```

80
81 %% NOZZLE
82 % Define nozzle parameters
83 mf = 0.0682 * m2_dot; % Mass flow rate (kg/s)
84 nozzle_expansion_angle = 3; % Expansion angle of the nozzle (degrees)
85 M4 = M4_array(2); % Mach number at nozzle entrance
86 discrete_lengths = linspace(0.005, 0.5, 100); % Discrete lengths along
    the nozzle (m)
87 initial_nozzle_height = 50e-3 + (2 * tand(combustor_expansion_angle) *
    1); % Initial height of the nozzle (m)
88 initial_nozzle_width = 70e-3; % Initial width of the nozzle (m)
89
90 % Calculate total pressure at the nozzle entrance
91 Pt4 = P4 * (1 + ((gamma_combustor - 1) / 2) * M4^2)^(gamma_combustor /
    (gamma_combustor - 1));
92 gamma_nozzle = gamma_combustor; % Heat capacity ratio in the nozzle
93
94 % Calculate nozzle flow properties and thrust
95 [nozzle_thrust, A9, M_bar, M_tests] = nozzle1(A4, gamma_nozzle, Pt4, M4
    , initial_nozzle_height, initial_nozzle_width, discrete_lengths,
    nozzle_expansion_angle);
96
97 % Display the total nozzle thrust
98 fprintf('Total Nozzle thrust for %s: %.2f N\n', dist_name,
    nozzle_thrust);
99
100 % Calculate total thrust and specific impulse
101 total_thrust = nozzle_thrust + combustor_thrust;
102 specific_impulse_g = total_thrust / (mf * 9.81);
103 specific_impulse = total_thrust / mf;
104
105 % Display final results
106 fprintf('
    -----\n
    ');
107 fprintf('
    -----\n
    ');
108 fprintf('Total thrust for %s: %.2f N\n', dist_name, total_thrust);
109 fprintf('Specific Impulse with g for %s: %.2f N/kg/s\n', dist_name,
    specific_impulse_g);
110 fprintf('Specific Impulse without g for %s: %.2f N/kg/s\n', dist_name,
    specific_impulse);

```

A.2 Freestream Conditions

```

1 function [T_infty, P_infty] = FreestreamConditions1(h)
2     % Function to calculate freestream static temperature (T_infty) and
3     % static pressure (P_infty) based on flight altitude (h).
4     % Inputs:
5     %   h - flight altitude in meters
6     % Outputs:
7     %   T_infty - freestream static temperature in Kelvin
8     %   P_infty - freestream static pressure in Pascals

```

```

9
10 % Check if the altitude is within the valid range
11 if (h < 0) || (h > 3e4)
12     % Print error message and set outputs to NaN if altitude is out
        of range
13     fprintf('The flight altitude is outside of the range\n')
14     T_infty = NaN;
15     P_infty = NaN;
16
17 % Calculate T_infty and P_infty for altitude below 11,000 meters
18 elseif h < 11000
19     T_infty = 288.19 - (0.00649 * h);
20     P_infty = 1.01e5 * (T_infty / 288.19)^5.256;
21
22 % Calculate T_infty and P_infty for altitude between 11,000 and
        25,000 meters
23 elseif (11000 <= h) && (h < 25000)
24     T_infty = 216.69;
25     P_infty = 2.27e4 * exp(1)^(1.73 - (0.000157 * h));
26
27 % Calculate T_infty and P_infty for altitude above 25,000 meters
28 else
29     T_infty = 141.94 + (0.00299 * h);
30     P_infty = 2.49e3 * (T_infty / 216.6)^-11.388;
31 end
32 end

```

A.3 Stagnation Conditions

```

1 function [Tt_infty, Pt_infty] = StagnationConditions1(T_infty, P_infty,
        M_infty, gamma_infty)
2     % Function to calculate freestream stagnation temperature (Tt_infty
        ) and
3     % stagnation pressure (Pt_infty) based on freestream conditions and
        Mach number.
4     % Inputs:
5     % T_infty - freestream static temperature in Kelvin
6     % P_infty - freestream static pressure in Pascals
7     % M_infty - freestream Mach number
8     % gamma_infty - freestream heat capacity ratio
9     % Outputs:
10    % Tt_infty - freestream stagnation temperature in Kelvin
11    % Pt_infty - freestream stagnation pressure in Pascals
12
13    % Calculate freestream stagnation temperature (Tt_infty)
14    Tt_infty = (1 + ((gamma_infty - 1) / 2 * M_infty^2)) * T_infty;
15
16    % Calculate freestream stagnation pressure (Pt_infty)
17    Pt_infty = (1 + ((gamma_infty - 1) / 2 * M_infty^2))^(gamma_infty /
        (gamma_infty - 1)) * P_infty;
18 end

```

A.4 Inlet

```

1 function [inlet_Mach_array, inlet_temps, inlet_pressures, temp_ratio,
   pressure_ratio, beta] = inlet1(theta_array, M_infty, gamma_infty,
   beta_array, T_infty, P_infty)
2   % Function to calculate the flow properties downstream of a four
   oblique shock inlet.
3   % Inputs:
4   %   theta_array - array of deflection angles for the ramps
5   %   M_infty - freestream Mach number
6   %   gamma_infty - freestream heat capacity ratio
7   %   beta_array - array of shock angles
8   %   T_infty - freestream static temperature
9   %   P_infty - freestream static pressure
10  % Outputs:
11  %   inlet_Mach_array - array of Mach numbers at each shock
12  %   inlet_temps - array of temperatures at each shock
13  %   inlet_pressures - array of pressures at each shock
14  %   temp_ratio - temperature ratio across each shock
15  %   pressure_ratio - pressure ratio across each shock
16  %   beta - shock angles for each ramp
17
18  % Initialize freestream conditions for the first shock
19  M = M_infty;
20  T = T_infty;
21  P = P_infty;
22
23  % Loop through each deflection angle to calculate flow properties
24  for i = 1:length(theta_array)
25      % Calculate possible shock angles using tan function
26      numerator = (M^2 * (sind(beta_array).^2)) - 1;
27      denominator = (M^2 * (gamma_infty + cosd(2 .* beta_array))) +
          2;
28      tantheta = 2 .* cotd(beta_array) .* (numerator ./ denominator);
29
30      possible_theta = atand(tantheta);
31      possible_theta(possible_theta < 0) = NaN;
32
33      % Check if deflection angle exceeds maximum possible value
34      if theta_array(i) > max(possible_theta)
35          % Set flow properties to NaN and display error message
36          T(i) = NaN;
37          p(i) = NaN;
38          sprintf('Theta, %.f, is greater than the maximum deflection
              angle at this Mach number, %.f', theta_array(i), M)
39      else
40          %% Calculating exact value of beta for weak shock solutions
41          delta = 1; % Weak shock solutions
42          term1 = (M^2 - 1)^2;
43          term2 = 3 * (1 + ((gamma_infty - 1) / 2) * M^2);
44          term3 = (1 + ((gamma_infty + 1) / 2) * M^2) * (tand(
              theta_array(i)))^2;
45          lambda = sqrt(term1 - (term2 * term3));
46
47          term4 = (M^2 - 1)^3;
48          term5 = 9 * (1 + ((gamma_infty - 1) / 2) * M^2);
49          term6 = (1 + ((gamma_infty - 1) / 2) * M^2 + ((gamma_infty
              + 1) / 4) * M^4) * tand(theta_array(i))^2;

```

```

50     chi = (term4 - (term5 * term6)) / lambda^3;
51
52     term7 = (4 * pi * delta + acos(chi)) / 3;
53     numerator = M^2 - 1 + (2 * lambda * cos(term7));
54     denominator = 3 * (1 + ((gamma_infty - 1) / 2) * M^2) *
        tand(theta_array(i));
55
56     beta(i) = atand(numerator / denominator);
57
58     %% Ratios and Flow Properties
59
60     % Mach Number
61     Mn = M * sind(beta(i));
62     M2n = sqrt((1 + ((gamma_infty - 1) / 2) * Mn^2) / (
        gamma_infty * Mn^2 - ((gamma_infty - 1) / 2)));
63     inlet_Mach_array(i) = M2n / sind(beta(i) - theta_array(i));
64     M = inlet_Mach_array(i); % Update M for the next loop
65
66     % Temperature Ratio
67     part1 = 1 + ((2 * gamma_infty) / (gamma_infty + 1)) * (Mn^2
        - 1);
68     part2 = (2 + (gamma_infty - 1) * Mn^2) / ((gamma_infty + 1)
        * Mn^2);
69     temp_ratio(i) = part1 * part2;
70     inlet_temps(i) = temp_ratio(i) * T;
71     T = inlet_temps(i); % Update T for the next loop
72
73     % Pressure Ratio
74     pressure_ratio(i) = 1 + ((2 * gamma_infty) / (gamma_infty +
        1)) * (Mn^2 - 1);
75     inlet_pressures(i) = pressure_ratio(i) * P;
76     P = inlet_pressures(i); % Update P for the next loop
77
78     end
79 end

```

A.5 Isolator

```

1 function [M2, T2, P2, m2_dot] = isolator1(iso_pt_ratio, iso_p_ratio, M1
    , T1, P1, gamma_isolator, iso_area, R)
2     % Function to calculate the flow properties at the isolator exit
3     % Inputs:
4     %     iso_pt_ratio - Total pressure recovery ratio (p_t2 / p_t1)
5     %     iso_p_ratio - Static pressure ratio (p2 / p1)
6     %     M1 - Mach number at isolator entrance
7     %     T1 - Static temperature at isolator entrance (K)
8     %     P1 - Static pressure at isolator entrance (Pa)
9     %     gamma_isolator - Heat capacity ratio at isolator
10    %     iso_area - Cross-sectional area of isolator (m^2)
11    %     R - Specific gas constant for air (J/(kgK))
12    % Outputs:
13    %     M2 - Mach number at isolator exit
14    %     T2 - Static temperature at isolator exit (K)
15    %     P2 - Static pressure at isolator exit (Pa)

```



```

16 % m2_dot - Mass flow rate at isolator exit (kg/s)
17
18 % Calculate total pressure at isolator entrance
19 Pt1 = P1 * (1 + (gamma_isolator - 1) / 2 * M1^2)^(gamma_isolator /
20 (gamma_isolator - 1));
21
22 % Calculate total pressure at isolator exit
23 Pt2 = Pt1 * iso_pt_ratio;
24
25 % Calculate static pressure at isolator exit
26 P2 = P1 * iso_p_ratio;
27
28 % Calculate Mach number at isolator exit
29 M2 = sqrt((2 / (gamma_isolator - 1)) * ((Pt2 / P2)^((gamma_isolator
30 - 1) / gamma_isolator) - 1));
31
32 % Calculate static temperature at isolator exit
33 T2 = T1 * (1 + (gamma_isolator - 1) / 2 * M1^2) / (1 + (
34 gamma_isolator - 1) / 2 * M2^2);
35
36 % Calculate density at isolator exit
37 rho2 = P2 / (R * T2);
38
39 % Calculate velocity at isolator exit
40 V2 = M2 * sqrt(gamma_isolator * R * T2);
41
42 % Calculate mass flow rate at isolator exit
43 m2_dot = rho2 * iso_area * V2;
44 end

```

A.6 Combustor

```

1 function [combustor_thrust, P4, A4] = combustor1(P2, com_p_ratios,
2 com_p_ratio_lengths, initial_combustor_height,
3 initial_combustor_width, combustor_expansion_angle)
4 % Function to calculate the thrust generated by the combustor
5 % Inputs:
6 % P2 - Static pressure at combustor entrance (Pa)
7 % com_p_ratios - Array of pressure ratios along the combustor
8 % com_p_ratio_lengths - Array of distances along the combustor (m)
9 % initial_combustor_height - Initial height of the combustor (m)
10 % initial_combustor_width - Initial width of the combustor (m)
11 % combustor_expansion_angle - Expansion angle of the combustor (
12 degrees)
13 % Outputs:
14 % combustor_thrust - Total thrust generated by the combustor (N)
15 % P4 - Static pressure at combustor exit (Pa)
16 % A4 - Cross-sectional area at combustor exit (m^2)
17
18 % Initialize total combustor thrust
19 combustor_thrust = 0;

```

```

18 % Loop through each pressure ratio length to calculate thrust
    contribution
19 for i = 1:length(com_p_ratio_lengths)
20     % Calculate the segment length
21     if i == 1
22         dx = com_p_ratio_lengths(i);
23     else
24         dx = com_p_ratio_lengths(i) - com_p_ratio_lengths(i - 1);
25     end
26
27     % Calculate pressure at the current segment
28     p_bar = com_p_ratios(i) * P2;
29
30     % Effective area of the segment
31     a_eff = initial_combustor_width * 2 * tand(
        combustor_expansion_angle) * dx;
32
33     % Add the segment thrust contribution to the total combustor
        thrust
34     combustor_thrust = combustor_thrust + (p_bar * a_eff);
35 end
36
37 % Calculate the exit area of the combustor
38 A4 = initial_combustor_width * (initial_combustor_height + (2 *
    tand(combustor_expansion_angle) * 1));
39
40 % Calculate the static pressure at the combustor exit
41 P4 = com_p_ratios(end) * P2;
42
43
44 end

```

A.7 Nozzle

```

1 function [nozzle_thrust, A9, M_bar, M_tests] = nozzle1(A4, gamma_nozzle
    , Pt4, M4, initial_nozzle_height, initial_nozzle_width,
    discrete_lengths, nozzle_expansion_angle)
2     % Function to calculate the thrust generated by the nozzle
3     % Inputs:
4     % A4 - Cross-sectional area at nozzle entrance (m^2)
5     % gamma_nozzle - Heat capacity ratio at nozzle
6     % Pt4 - Total pressure at nozzle entrance (Pa)
7     % M4 - Mach number at nozzle entrance
8     % initial_nozzle_height - Initial height of the nozzle (m)
9     % initial_nozzle_width - Initial width of the nozzle (m)
10    % discrete_lengths - Array of distances along the nozzle (m)
11    % nozzle_expansion_angle - Expansion angle of the nozzle (degrees
    )
12    % Outputs:
13    % nozzle_thrust - Total thrust generated by the nozzle (N)
14    % A9 - Cross-sectional area at nozzle exit (m^2)
15    % M_bar - Mach number at each discrete length (array)
16    % M_tests - Mach number solutions at each discrete length (array)
17

```

```

18 % Calculate the area ratio A/A*
19 area_ratio = (1 / M4^2) * ((2 / (gamma_nozzle + 1)) * (1 + ((
    gamma_nozzle - 1) / 2) * M4^2))^((gamma_nozzle + 1) / (
    gamma_nozzle - 1));
20
21 % Calculate A* (throat area)
22 A_star = A4 / sqrt(area_ratio);
23
24 % Initialize nozzle thrust
25 nozzle_thrust = 0;
26
27 % Loop through each discrete length to calculate thrust
    contribution
28 for i = 1:length(discrete_lengths)
29     % Calculate the segment length
30     if i == 1
31         dx = discrete_lengths(i);
32     else
33         dx = discrete_lengths(i) - discrete_lengths(i - 1);
34     end
35
36     % Calculate the area at the current segment
37     A_bar = initial_nozzle_width * ((2 * tand(
        nozzle_expansion_angle) * discrete_lengths(i)) +
        initial_nozzle_height);
38
39     % Effective area of the segment
40     A_eff = 2 * (initial_nozzle_width * tand(nozzle_expansion_angle
        ) * dx);
41
42     %% Solve for Mach Number
43     syms M_bar
44     A_ratio = A_bar / A_star;
45     eqn = A_ratio^2 == (1 / M_bar^2) * ((2 / (gamma_nozzle + 1)) *
        (1 + ((gamma_nozzle - 1) / 2) * M_bar^2))^((gamma_nozzle +
        1) / (gamma_nozzle - 1));
46
47     % Solve for Mach number
48     M_solution = vpasolve(eqn, M_bar, [1 10]);
49
50     % Convert symbolic solutions to numeric
51     M_solution_numeric = double(M_solution);
52
53     % Filter out complex solutions
54     real_solutions = M_solution_numeric(imag(M_solution_numeric) ==
        0);
55
56     % Find the largest real solution
57     M_bar = max(real_solutions);
58     M_tests(i) = max(real_solutions);
59
60     %% Thrust Contribution
61     P_eff = (((1 + ((gamma_nozzle - 1) / 2) * M_bar^2))^(-
        gamma_nozzle / (gamma_nozzle - 1))) * Pt4;
62
63     P_thrust = P_eff; % Uncomment if thrust needs to be adjusted by
        cosine factor
64     nozzle_thrust = nozzle_thrust + (P_thrust * A_eff);

```

```
65     end
66
67     % Calculate the exit area of the nozzle
68     A9 = initial_nozzle_width * (initial_nozzle_height + (2 * tand(
        nozzle_expansion_angle) * 0.5));
69 end
```